🛠️ TALLER PRÁCTICO EN EL AULA

Prácticas Guiadas
Objetos Literales en JS

Ejercicios paso a paso para comparar variables sueltas frente a objetos literales, propiedades dinámicas, métodos interactivos con `this` y gestión de colecciones mediante arrays de objetos.

🎮 Práctica 1: Ficha de Personaje

Transforma un conjunto disperso de variables globales en un héroe autónomo con salud, nivel y métodos interactivos (`recibirDanio`, `curar`, `subirNivel`).

`Propiedades · Métodos · this`

🛒 Práctica 2: Cesta de la Compra

Sustituye arrays paralelos desincronizados por una colección de objetos literales con cálculo automático de subtotales por artículo y total acumulado.

`Arrays de Objetos · for…of · subtotal()`

Profesor Ramón Abramo
Especialista en Programación Web Frontend · Más de 25 años de experiencia docente

Edición Académica 2026
Santander · España

📑 ÍNDICE GENERAL DE LAS PRÁCTICAS

(Haz clic en cualquier práctica para saltar a ella)

- 📦 Portada Oficial del Taller — Pág. 1
- 📋 Instrucciones Generales para el Alumno — Pág. 2
- 📊 Comparativa: Variables sueltas vs Objetos Literales — Pág. 2
- 🎮 Práctica 1: Ficha de Personaje (Enunciado y Base) — Pág. 3
- 🧭 Práctica 1: Guía de Desarrollo y Solución — Pág. 4
- 🛒 Práctica 2: Carrito de la Compra (Enunciado y Base) — Pág. 5
- 🧭 Práctica 2: Guía de Desarrollo y Solución — Pág. 6
- ✅ Gran Rúbrica de Autoevaluación del Alumno — Pág. 6

📋 Instrucciones Generales para el Alumnado

🎯 Metodología de Trabajo

En ambas prácticas dispones del HTML y el CSS completamente maquetados, junto a un archivo `app.js` con comentarios guía numerados paso a paso. Tu tarea consiste en programar exclusivamente la lógica JavaScript sin modificar los identificadores (`id`) existentes en el HTML.

📊 Comparativa Didáctica: ¿Por qué usar Objetos Literales?

CRITERIO DIDÁCTICO	CON VARIABLES O ARRAYS SUELTOS	CON OBJETOS EN NOTACIÓN LITERAL
Organización del Código	Datos dispersos (`nombre` , `vida` , `nivel` por separado) propensos a desincronizarse o colisionar de nombre.	Todo viaja unido bajo una única entidad coherente (`personaje` o cada `nuevoProducto`).
Comportamiento Propio	Las funciones están sueltas y obligan a recibir todos los datos como parámetros externos.	Los métodos viven dentro del propio objeto y acceden a su estado interno mediante `this` .
Escalabilidad en Listas	Obliga a mantener 3 o más arrays paralelos (`nombres[i]` , `precios[i]`) con el mismo índice numérico.	Un único array de objetos (`carrito`) donde cada posición es una entidad completa y autónoma.
Legibilidad y Mantenimiento	Código confuso que requiere recordar qué posición numérica corresponde a cada dato.	Propiedades autoexplicativas con nombre descriptivo (`prod.precio` , `prod.subtotal()`).

⚡ Regla de Oro en el Aula

Cuando un grupo de variables siempre se utilicen juntas para describir una misma entidad o registro, **debes agruparlas en un objeto literal**. Los arrays paralelos son un antipatrón en programación profesional.

🎮 Práctica 1: Ficha de Personaje (De variables sueltas a Objeto Literal)

Objetivo Didáctico: Modelar la entidad `personaje` de un videojuego en un objeto literal con datos y métodos internos con `this`, controlando la actualización reactiva de la interfaz.

> 📖 **Enunciado del Proyecto**
> El objeto `personaje` debe iniciar con las propiedades `nombre: "Valeria"`, `clase: "Exploradora"`, `nivel: 1` y `vida: 100`. Además, debe implementar 3 métodos: `recibirDanio(puntos)` (resta vida sin bajar de 0), `curar(puntos)` (suma vida sin pasar de 100) y `subirNivel()` (incrementa nivel y regenera vida al 100%). Cada método devuelve un mensaje que se muestra en `#registro`.

🧱 Código Base Entregado (HTML y CSS)

```html
<!-- Estructura de practica_01_personaje/index.html -->
<div class="ficha">
  <h2>🛡️ Ficha del Héroe</h2>
  <div class="dato">Nombre: <strong id="txtNombre">—</strong></div>
  <div class="dato">Clase: <strong id="txtClase">—</strong></div>
  <div class="dato">Nivel: <strong id="txtNivel">—</strong></div>
  <div class="dato">Puntos de Vida: <strong id="txtVida">—</strong></div>

  <div class="botones">
    <button id="btnDanio">⚔️ Daño (-20)</button>
    <button id="btnCurar">💉 Curar (+15)</button>
    <button id="btnNivel">⭐ Subir Nivel</button>
  </div>

  <p id="registro">Esperando acciones...</p>
</div>
```

📋 Guion Comentado en `app.js` para el Alumno

```javascript
const txtNombre = document.getElementById("txtNombre");
const txtClase  = document.getElementById("txtClase");
const txtNivel  = document.getElementById("txtNivel");
const txtVida   = document.getElementById("txtVida");
const registro  = document.getElementById("registro");

// PASO 2: Declara el objeto literal "personaje" con sus propiedades y métodos:
const personaje = {
  // TODO: Añade nombre, clase, nivel y vida inicial
  // TODO: Añade recibirDanio(puntos) → resta a this.vida (mín 0)
  // TODO: Añade curar(puntos) → suma a this.vida (máx 100)
  // TODO: Añade subirNivel() → incrementa this.nivel y restaura this.vida = 100
};
```

🧭 Guía Paso a Paso y Solución Oficial Comentada

Paso 1: Captura con `getElementById` los elementos de la ficha (`txtNombre` , `txtVida` , etc.) y los botones.

Paso 2: Define el objeto `personaje` . Dentro de los métodos usa `this.vida` y comprueba los límites (`this.vida < 0` o `> 100`).

Paso 3: Escribe `actualizarFicha()` asignando `personaje.nombre` a `txtNombre.textContent` y así sucesivamente.

Paso 4: En los eventos `click` , llama al método correspondiente, guarda el mensaje devuelto en `registro.textContent` y refresca con `actualizarFicha()` .

💻 Solución Oficial del Profesor (`app.js`)

```
const personaje = {
  nombre: "Valeria",
  clase: "Exploradora",
  nivel: 1,
  vida: 100,

  recibirDanio(puntos) {
    if (this.vida <= 0) return `¡${this.nombre} ya está fuera de combate!`;
    this.vida -= puntos;
    if (this.vida < 0) this.vida = 0;
    return `⚔️ ¡${this.nombre} recibe ${puntos} pts de daño! Salud: ${this.vida}/100`;
  },

  curar(puntos) {
    if (this.vida <= 0) return `❌ ¡${this.nombre} está derrotada y no puede curarse!`;
    this.vida += puntos;
    if (this.vida > 100) this.vida = 100;
    return `🧪 ¡${this.nombre} se cura ${puntos} pts! Salud: ${this.vida}/100`;
  },

  subirNivel() {
    this.nivel += 1;
    this.vida = 100;
    return `⭐ ¡Subida de nivel! ${this.nombre} alcanza el Nivel ${this.nivel}.`;
  }
};

function actualizarFicha() {
  txtNombre.textContent = personaje.nombre;
  txtClase.textContent  = personaje.clase;
  txtNivel.textContent  = `Nivel ${personaje.nivel}`;
  txtVida.textContent   = `${personaje.vida} / 100`;
}

btnDanio.addEventListener("click", () => {
  registro.textContent = personaje.recibirDanio(20);
  actualizarFicha();
});

btnCurar.addEventListener("click", () => {
  registro.textContent = personaje.curar(15);
  actualizarFicha();
});

btnNivel.addEventListener("click", () => {
  registro.textContent = personaje.subirNivel();
  actualizarFicha();
});

actualizarFicha(); // Carga inicial
```

🛒 Práctica 2: Carrito de la Compra con Objetos Literales

Objetivo Didáctico: Gestionar una colección de entidades dinámicas mediante un único array `carrito` donde cada elemento es un objeto literal con propiedades y un método propio de cálculo `subtotal()`.

📖 Enunciado de la Práctica

Al rellenar el formulario (producto, precio y cantidad) y hacer clic en *"Añadir al carrito"*, debes construir un nuevo objeto literal con dichos valores y un método `subtotal() { return this.precio * this.cantidad; }`. El objeto se añadirá a `carrito` con `.push()` y se invocará `dibujarCarrito()` para iterar el array con `for ... of` y calcular el total acumulado.

🧱 Código Base Entregado (HTML y CSS)

```
<!-- Estructura de practica_02_carrito/index.html -->
<div class="caja-carrito">
  <h2>🛒 Cesta de la Compra</h2>
  <div class="fila-inputs">
    <input type="text" id="inpNombre" placeholder="Producto">
    <input type="number" id="inpPrecio" placeholder="Precio (€)">
    <input type="number" id="inpCantidad" value="1">
  </div>
  <button id="btnAgregar">➕ Añadir al carrito</button>
  <ul id="listaCarrito"></ul>
  <div class="bloque-total">Total: <strong id="txtTotal">0.00</strong> €</div>
</div>
```

📋 Plantilla Comentada de `app.js`

```
const inpNombre    = document.getElementById("inpNombre");
const inpPrecio    = document.getElementById("inpPrecio");
const inpCantidad  = document.getElementById("inpCantidad");
const btnAgregar   = document.getElementById("btnAgregar");
const listaCarrito = document.getElementById("listaCarrito");
const txtTotal     = document.getElementById("txtTotal");

const carrito = []; // Array que albergará los objetos literales
```

🧭 Guía Paso a Paso y Solución Oficial Comentada

Paso 1: Al pulsar `btnAgregar`, lee los valores con `parseFloat` y valida que no estén vacíos.

Paso 2: Crea `nuevoProducto = { nombre, precio, cantidad, subtotal() { return this.precio * this.cantidad; } }`.

Paso 3: Insértalo con `carrito.push(nuevoProducto)`, limpia los campos y llama a `dibujarCarrito()`.

Paso 4: En `dibujarCarrito()`, limpia la lista, usa `for (const prod of carrito)`, crea `li` y acumula `prod.subtotal()`.

💻 Solución Oficial del Profesor (`app.js`)

```
btnAgregar.addEventListener("click", () => {
  const nombre   = inpNombre.value.trim();
  const precio   = parseFloat(inpPrecio.value);
  const cantidad = parseInt(inpCantidad.value, 10);

  if (!nombre || isNaN(precio) || precio <= 0 || isNaN(cantidad) || cantidad <= 0) {
    alert("Por favor, introduce datos válidos.");
    return;
  }

  const nuevoProducto = {
    nombre,
    precio,
    cantidad,
    subtotal() {
      return this.precio * this.cantidad;
    }
  };

  carrito.push(nuevoProducto);
  inpNombre.value = ""; inpPrecio.value = ""; inpCantidad.value = "1";
  dibujarCarrito();
});

function dibujarCarrito() {
  listaCarrito.innerHTML = "";
  let total = 0;

  for (const prod of carrito) {
    const li = document.createElement("li");
    li.innerHTML = `<span>${prod.nombre} (x${prod.cantidad})</span><strong>${prod.subtotal().toFixed(2)} €</strong>`;
    listaCarrito.appendChild(li);
    total += prod.subtotal();
  }

  txtTotal.textContent = total.toFixed(2);
}
```

✅ Rúbrica de Verificación y Autoevaluación

ÍTEM DE EVALUACIÓN	CRITERIO TÉCNICO ESPERADO	ESTADO
Práctica 1 · Encapsulación	Propiedades y métodos reunidos en el objeto literal `personaje` sin variables externas.	[ ] Superado
Práctica 1 · Uso de this	Los métodos usan `this.vida` y `this.nivel` evitando `ReferenceError`.	[ ] Superado
Práctica 2 · Array de Objetos	Se usa un único array `carrito` descartando los arrays paralelos.	[ ] Superado
Práctica 2 · Método subtotal()	Cada objeto calcula su propio coste con `this.precio * this.cantidad`.	[ ] Superado